Assignment 3 - MDA9159

Instructor: Dr. Guowen Huang

Fall 2025

-Student name: Rui Deng

-Student number: 251509628

Question 1

Weighted Least Squares (WLS)

Generate heteroscedastic data: wls_dat

```
library(MASS)
library(car)
```

```
## Warning: package 'car' was built under R version 4.5.2
```

```
## Loading required package: carData
```

```
## Warning: package 'carData' was built under R version 4.5.2
```

```
library(leaps)
```

```
## Warning: package 'leaps' was built under R version 4.5.2
```

```
library(glmnet)
```

```
## Warning: package 'glmnet' was built under R version 4.5.2
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```r
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.5.2
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

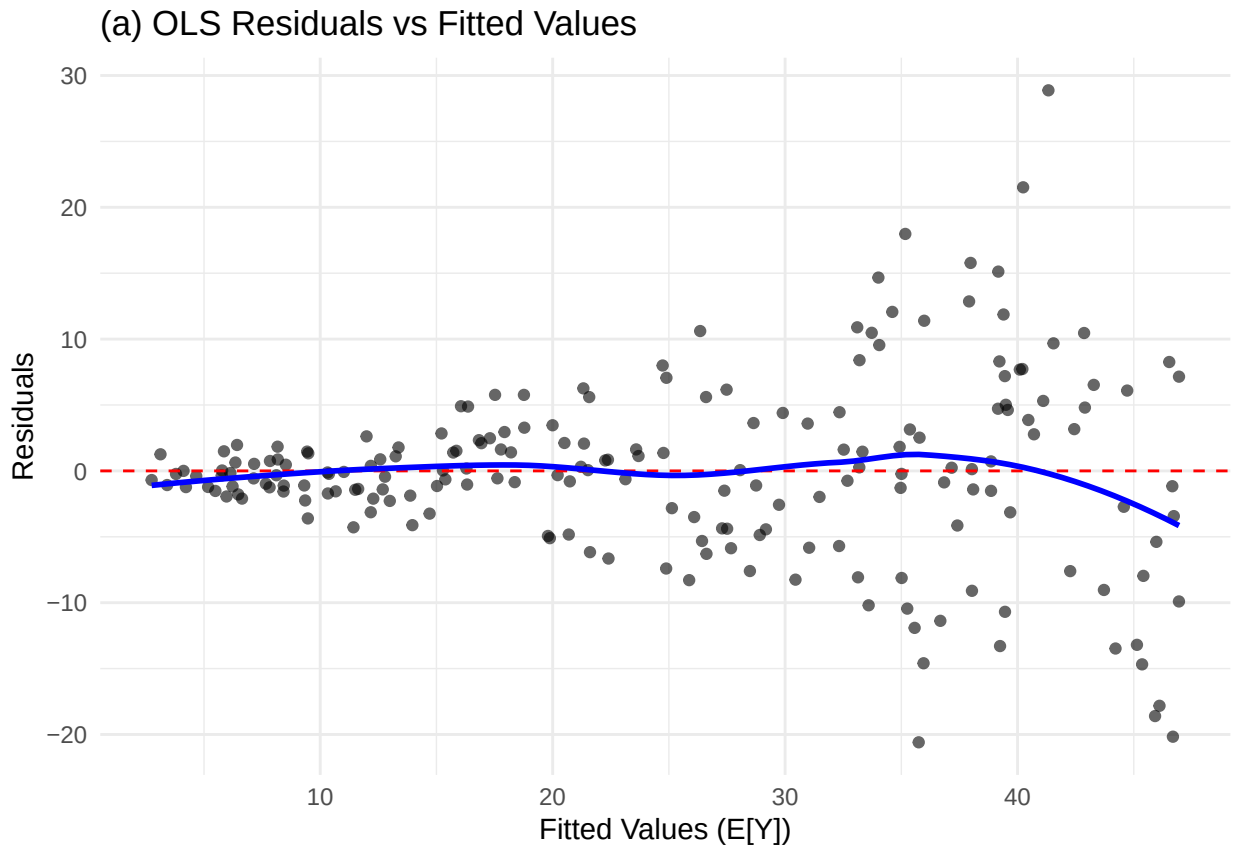
```r
library(ggplot2)
set.seed(9159)
n <- 200
x <- runif(n, 0, 10)
sigma2 <- x^2+runif(n)
y <- 1 + 4.8*x + rnorm(n, sd = sqrt(sigma2))
wls_dat <- data.frame(y, x)
```

(a) Fit OLS y ~ x. Plot residuals vs fitted values and comment on heteroscedasticity.

```r
ols_model <- lm(y ~ x, data = wls_dat)
plot1 <- ggplot(wls_dat, aes(x = fitted(ols_model), y = residuals(ols_model))) +
  geom_point(alpha = 0.6) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  geom_smooth(se = FALSE, color = "blue", method = "loess")+
  labs(title = "(a) OLS Residuals vs Fitted Values",
       x = "Fitted Values (E[Y])",
       y = "Residuals") +
  theme_minimal()

print(plot1)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



The plot of residuals versus fitted values exhibits a pronounced funnel shape. The vertical spread of the residuals clearly increases as the fitted values increase. This pattern violates the OLS assumption of homoscedasticity, meaning there might be heteroscedasticity.

(b) Fit WLS with weights wi $= 1/x^2$. Compare OLS and WLS coefficients, SEs, and CIs.

```r
wls_correct_model <- lm(y ~ x, data = wls_dat, weights = 1 / (wls_dat$x^2))
results_summary <- function(model, name) {
  s <- summary(model)$coefficients
  ci <- confint(model, level = 0.95)
  data.frame(
    Model = name,
    Term = rownames(s),
    Estimate = s[, 1],
    SE = s[, 2],
    CI_Lower = ci[, 1],
    CI_Upper = ci[, 2],
    row.names = NULL
  )
```

```
}

ols_res <- results_summary(ols_model, "OLS")
wls_res <- results_summary(wls_correct_model, "WLS (Correct)")
comparison_table <- rbind(ols_res, wls_res)
print(comparison_table)
```

```
##            Model        Term Estimate        SE   CI_Lower CI_Upper
## 1            OLS (Intercept) 1.635594 0.9908107 -0.3183020 3.589490
## 2            OLS           x 4.613749 0.1709489  4.2766349 4.950863
## 3 WLS (Correct) (Intercept) 1.258049 0.1632993  0.9360201 1.580078
## 4 WLS (Correct)           x 4.685884 0.1017596  4.4852121 4.886555
```

Estimates: WLS estimates are generally closer to the true population parameters ($\beta_0 = 1, \beta_1 = 4.8$) than OLS.

Standard Errors: The WLS model yields significantly smaller SEs for both coefficients compared to OLS. This is because WLS is the Best Linear Unbiased Estimator (BLUE) when heteroscedasticity is correctly accounted for, thus achieving the minimum variance among unbiased estimators.

Confidence Intervals: The CIs for WLS are narrower than OLS CIs, reflecting the increased precision of the WLS estimates.

(c) plot the regression lines from both OLS and WLS on the same scatter plot of the data and their prediction confidence intervals (in R: interval="prediction"), and comment on differences.

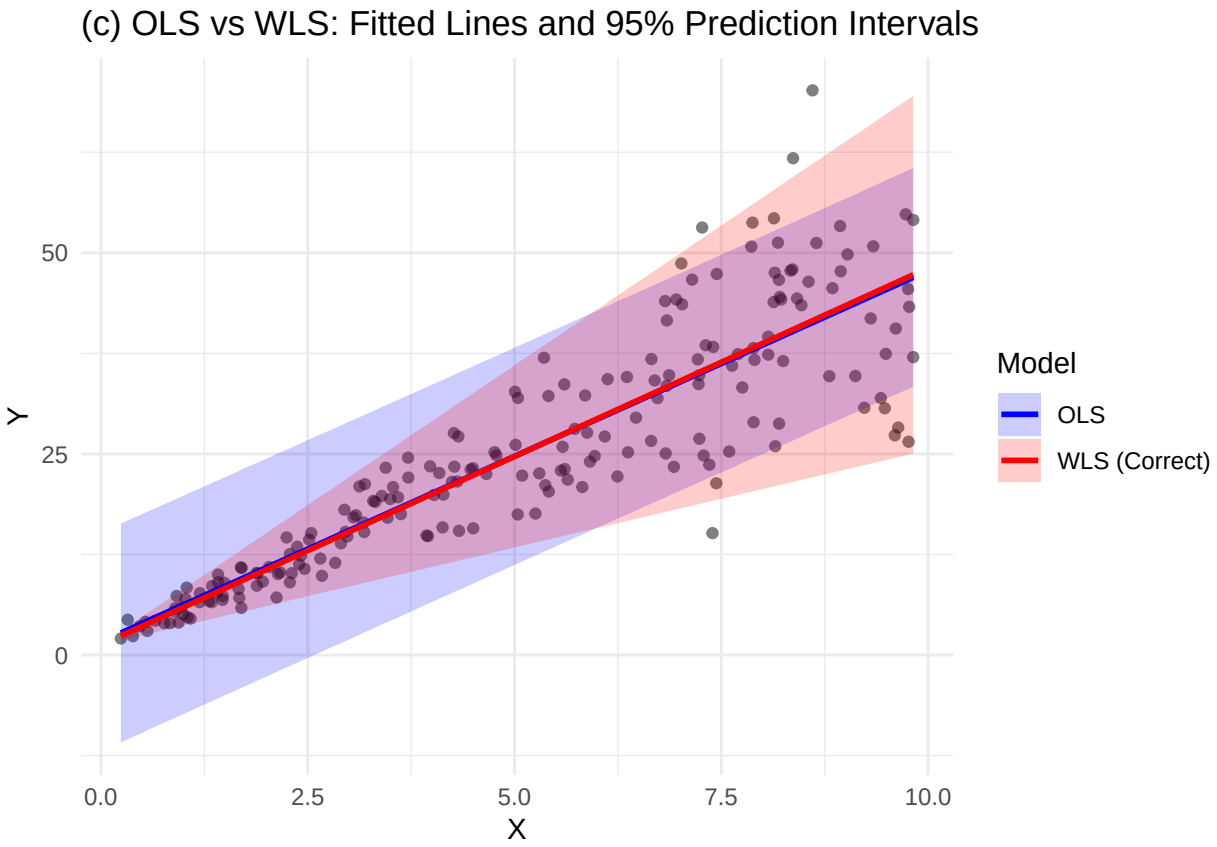
```
new_data <- data.frame(x = seq(min(wls_dat$x),
                               max(wls_dat$x),
                               length.out = 100))
ols_pred <- predict(ols_model,
                    newdata = new_data,
                    interval = "prediction")
ols_pred_df <- cbind(new_data,
                     ols_pred,
                     Model = "OLS")
wls_pred <- predict(wls_correct_model,
                    newdata = new_data,
                    interval = "prediction",
                    weights = 1 / (new_data$x^2))
wls_pred_df <- cbind(new_data,
```

```
                      wls_pred,
                      Model = "WLS (Correct)")

combined_pred_df <- rbind(ols_pred_df,
                          wls_pred_df)

plot_c <- ggplot(wls_dat,
                 aes(x = x, y = y)) +
  geom_point(alpha = 0.5) +
  geom_ribbon(data = combined_pred_df,
              aes(ymin = lwr,
                  ymax = upr,
                  fill = Model),
              alpha = 0.2) +
  geom_line(data = combined_pred_df,
            aes(y = fit, color = Model),
            linewidth = 1) +
  scale_fill_manual(values = c("OLS" = "blue", "WLS (Correct)" = "red")) +
  scale_color_manual(values = c("OLS" = "blue", "WLS (Correct)" = "red")) +
  labs(title = "(c) OLS vs WLS: Fitted Lines and 95% Prediction Intervals",
       x = "X",
       y = "Y") + theme_minimal()

print(plot_c)
```



Fitted Lines: The OLS and WLS fitted lines are typically very close, demonstrating that OLS still accurately captures the central trend, even under heteroscedasticity.

Prediction Intervals: The PIs show a critical difference: the OLS PI (blue region) has roughly constant width, failing to account for the increasing variance at higher X values. This makes OLS PIs too narrow at large X and too wide at small X. The WLS PI (red region) correctly widens as X increases, accurately reflecting the true, increasing variability of the data, as determined by the weights.

(d) Fit WLS with misspecified weights wi $= 1/(0.5 + 0.05x_i)^2$. Comment on the standard deviation of the estimate of the slope by comparing to OLS and WLS with correct-form of weights.

```
results_summary_df <- function(model, name) {
  s <- summary(model)$coefficients
  ci <- confint(model, level = 0.95)
  data.frame(
    Model = name,
    Term = rownames(s),
    Estimate = s[, 1],
```

```
    SE = s[, 2],
    CI_Lower = ci[, 1],
    CI_Upper = ci[, 2],
    row.names = NULL
  )
}
wls_miss_model <- lm(y ~ x, data = wls_dat, weights = 1 / (0.5 + 0.05 * wls_dat$x)^2)
miss_res <- results_summary_df(wls_miss_model, "WLS (Misspecified)")
se_comparison_df <- rbind(ols_res[ols_res$Term == "x", ],
                          wls_res[wls_res$Term == "x", ],
                          miss_res[miss_res$Term == "x", ])[,
print(se_comparison_df)
```

```
##                  Model Estimate        SE
## 2                  OLS 4.613749 0.1709489
## 21       WLS (Correct) 4.685884 0.1017596
## 22 WLS (Misspecified) 4.674465 0.1443625
```

The WLS (Correct) SE is the smallest, confirming its BLUE status. The WLS (Misspecified) SE is expected to be greater than the WLS (Correct) SE but potentially smaller than the OLS SE. This result confirms that some correction is better than none: because the misspecified weight function correctly captures the direction of heteroscedasticity , it provides a more efficient, albeit not optimally efficient, estimator than OLS. However, since the functional form of the weights is incorrect, its efficiency is lower than that of the optimally weighted WLS model.

Question 2

Correlated errors

One can load the data (on BrightSpace, week 11) by the command: p219 = read.table("P219.txt", h=T). The variables are - H: Housing Starts - P: Population Size of 22- to 45-yr-olds in millions - D: Availability for Mortgage Money Index

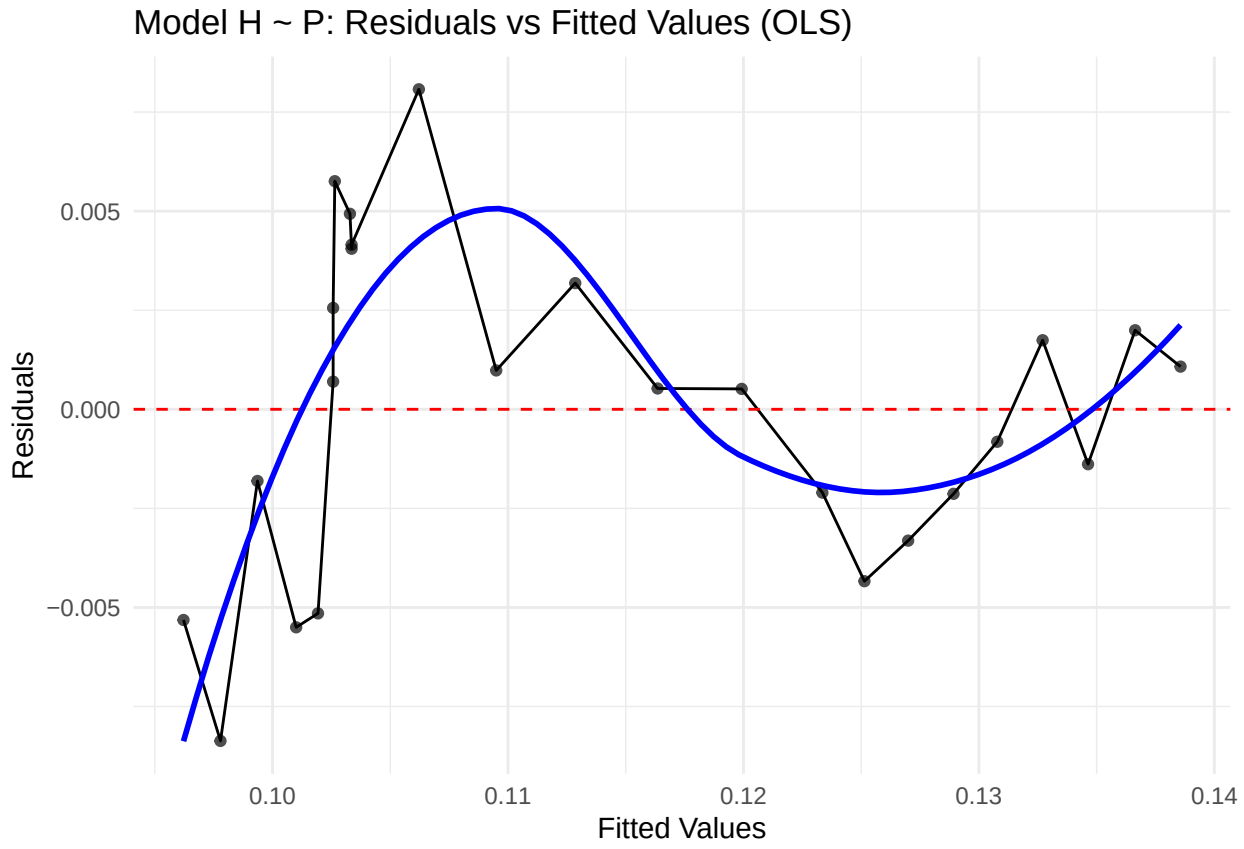
(a) Fit the linear model H ~ P. Plot residuals vs fitted values and comment on correlation structure of residuals.

```
p219 <- read.table("P219.txt", h = TRUE)
p219$Index <- 1:nrow(p219)
model1 <- lm(H ~ P, data = p219)
plot_a <- ggplot(p219, aes(x = fitted(model1), y = residuals(model1))) +
```

```
  geom_point(alpha = 0.7) +
  geom_line(group = 1) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  geom_smooth(se = FALSE, color = "blue", method = "loess") +
  labs(title = "Model H ~ P: Residuals vs Fitted Values (OLS)",
       x = "Fitted Values",
       y = "Residuals") +
  theme_minimal()

print(plot_a)
```

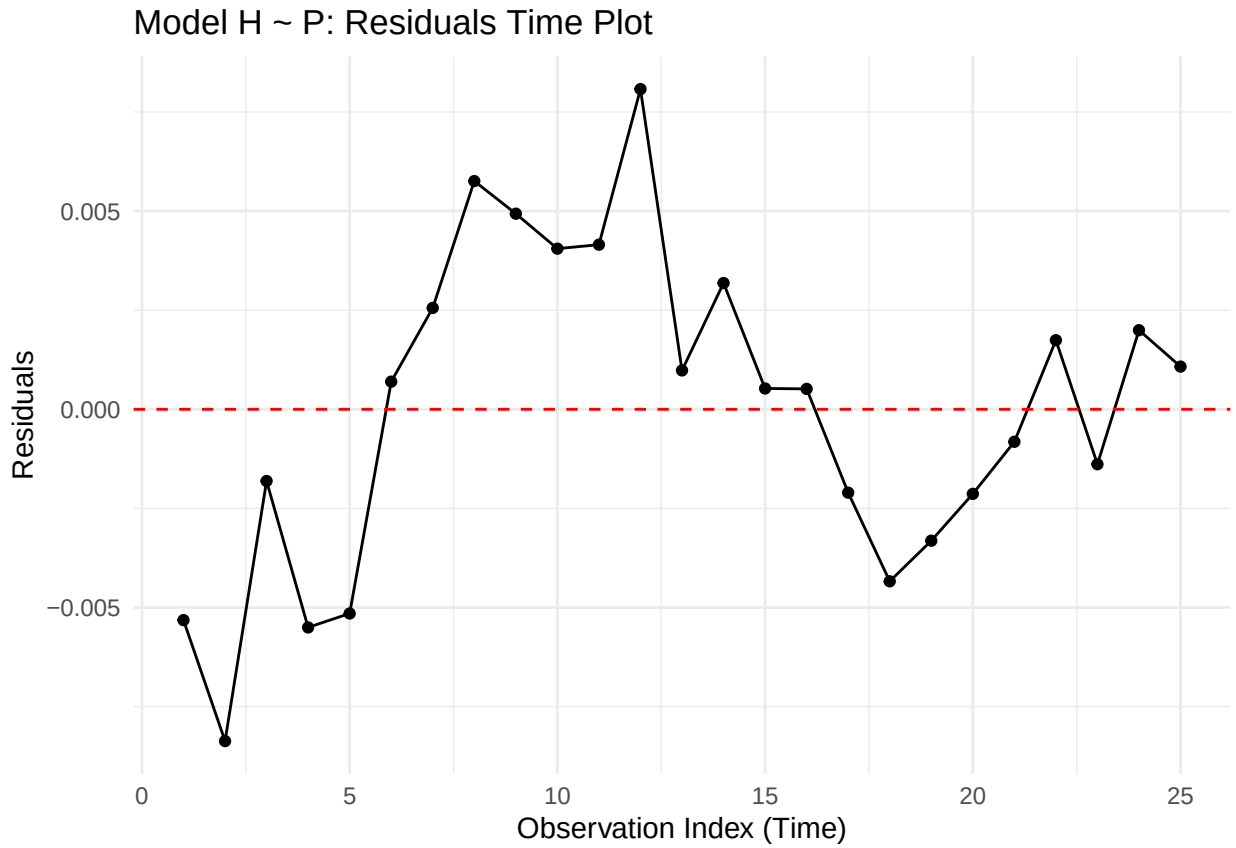
```
## `geom_smooth()` using formula = 'y ~ x'
```




```
plot_a_time <- ggplot(p219, aes(x = Index, y = residuals(model1))) +
  geom_point() +
  geom_line() +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  labs(title = "Model H ~ P: Residuals Time Plot",
       x = "Observation Index (Time)",
       y = "Residuals") +
```

```
  theme_minimal()

print(plot_a_time)
```



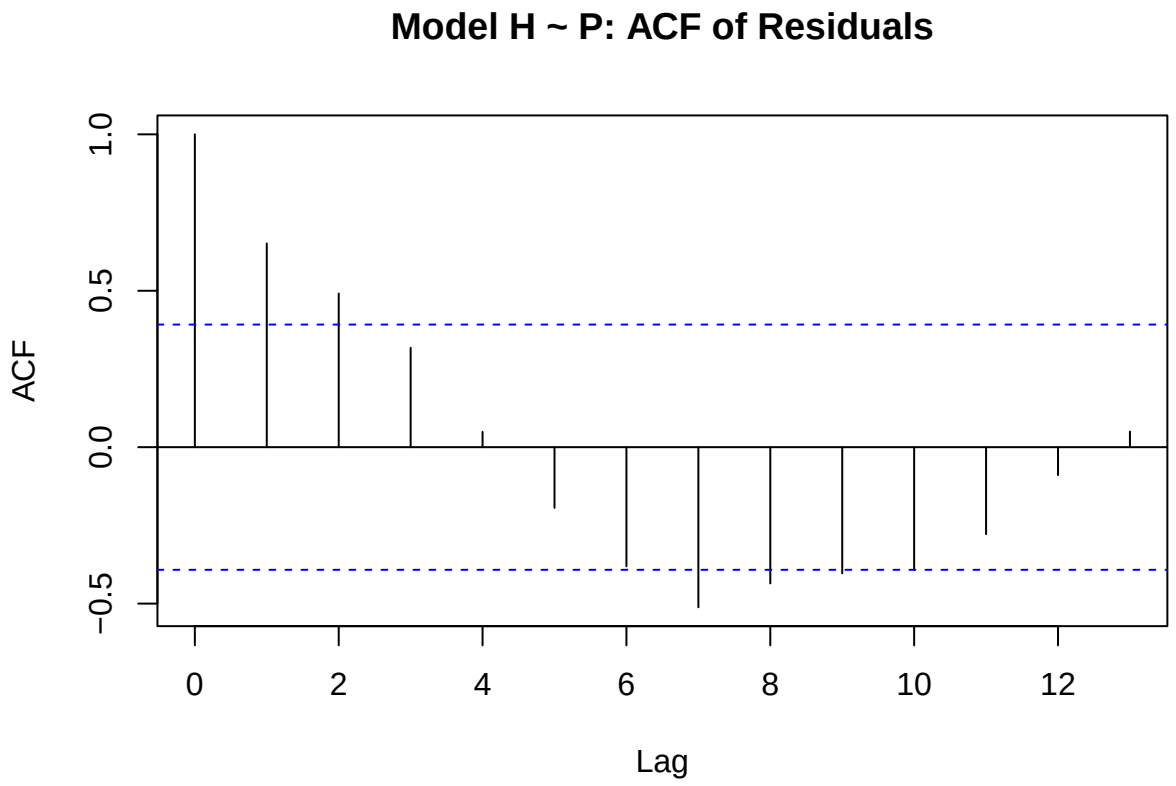
Pattern Recognition: The residuals do not scatter randomly around the zero line (red dashed line). Instead, they exhibit a clear smooth, wave-like pattern or an oscillating cycle.

Interpretation of Correlation: This smooth, cyclical pattern is the visual signature of strong positive autocorrelation. It shows that consecutive residuals are not independent; a positive residual tends to be followed by another positive residual, and a negative residual by another negative residual.

Conclusion: The residuals are clearly correlated over time, violating the assumption of independent errors in the OLS model.

(b) Plot ACF of residuals and comment on correlation structure.

```
acf_model1 <- acf(residuals(model1), main = "Model H ~ P: ACF of Residuals")
```



The bar at Lag 1 (the correlation between a residual e_t and its immediate predecessor e_{t-1}) extends significantly above the dashed blue confidence boundary2. This confirms the presence of strong first-order positive autocorrelation (ρ_1). The bars at Lag 2 and Lag 3 are also high, indicating that the correlation extends beyond immediately adjacent observations. The fact that the ACF values are significantly non-zero at multiple lags, particularly Lag 1, means the standard OLS assumption of independent errors is violated.Consequently, the standard errors of the OLS coefficients are likely underestimated , and the derived confidence intervals and significance tests are inaccurate.

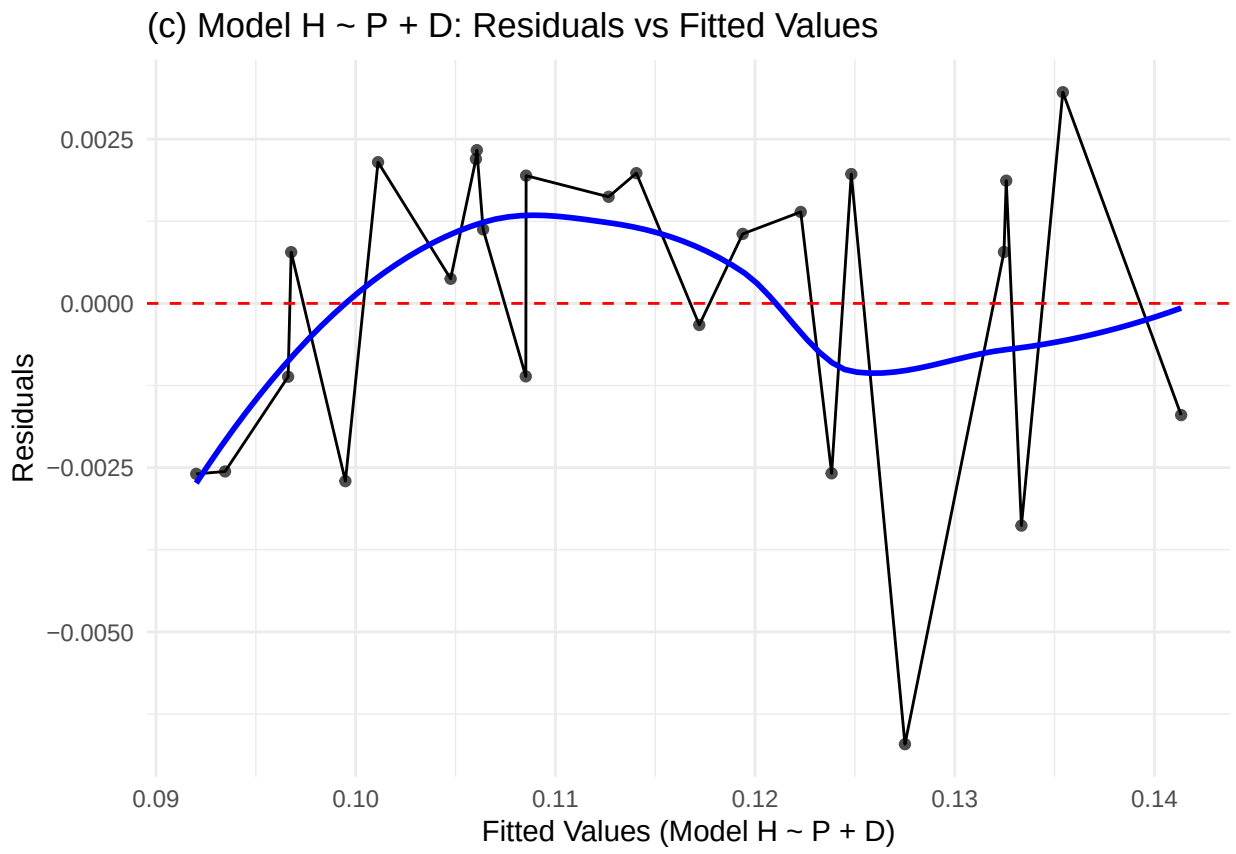
(c) Consider adding D as a predictor. Fit the model H ~ P + D. Plot residuals vs fitted values and ACF of residuals. Comment on whether adding D helps to mitigate correlation in errors.

```
model2 <- lm(H ~ P + D, data = p219)
plot_c_res <- ggplot(p219, aes(x = fitted(model2), y = residuals(model2))) +
  geom_point(alpha = 0.7) +
  geom_line(group = 1) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  geom_smooth(se = FALSE, color = "blue", method = "loess") +
  labs(title = "(c) Model H ~ P + D: Residuals vs Fitted Values",
```

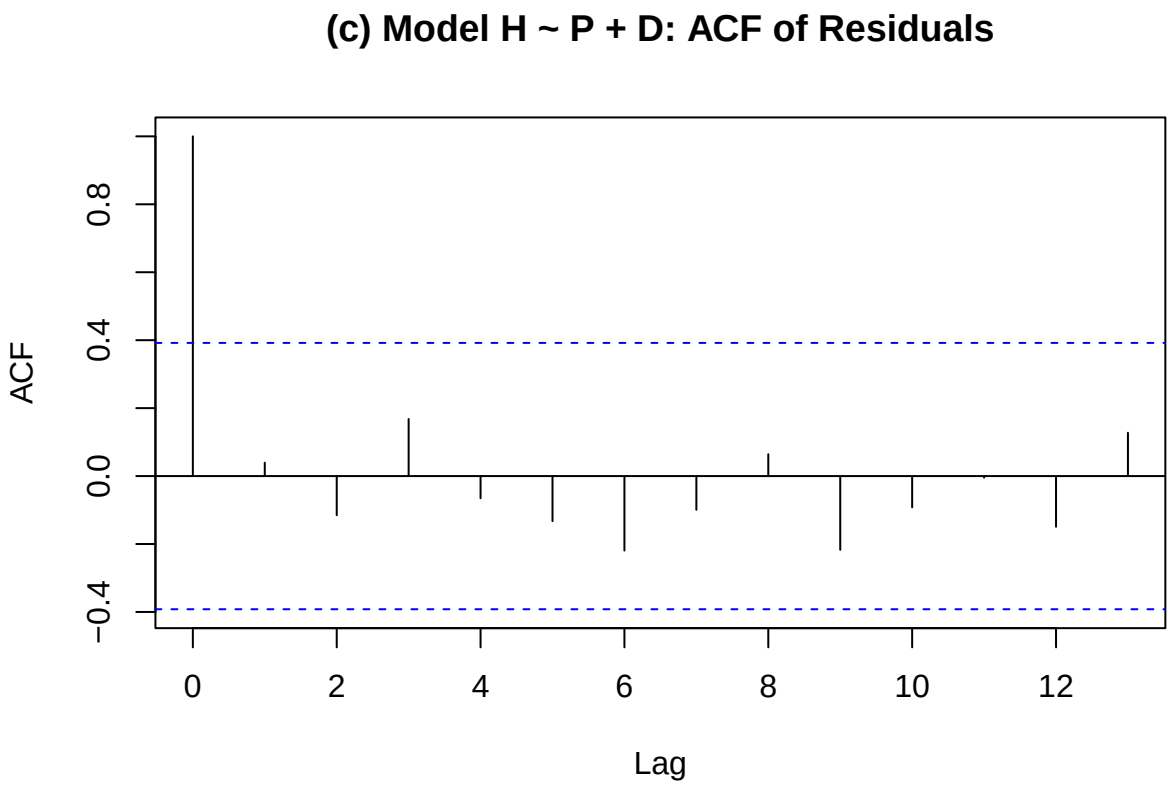
```
       x = "Fitted Values (Model H ~ P + D)",
       y = "Residuals") +
  theme_minimal()

print(plot_c_res)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```
acf_model2 <- acf(residuals(model2), main = "(c) Model H ~ P + D: ACF of Residuals")
```



Residuals vs. Fitted Values Plot Analysis: The plot shows a more random distribution of residuals around the red zero line compared to the initial model. While some short-term fluctuations persist (reflected by the blue smoother line), the clear, systematic, long-wave patterns observed in the $H \sim P$ model have been effectively eliminated.

ACF Plot Analysis: The ACF bar at Lag 1 falls well within the dashed blue confidence boundaries. The ACF values for all subsequent lags are also non-significant. The ACF plot definitively confirms that no statistically significant autocorrelation remains in the residuals of the $H \sim P + D$ model.

(d) Run a Durbin-Watson test on both models and comment on the results.

```
dw_model1 <- durbinWatsonTest(model1, alternative = "positive")
print(dw_model1)
```

```
##  lag Autocorrelation D-W Statistic p-value
##    1       0.6511468     0.6208403       0
##  Alternative hypothesis: rho > 0
```

```r
dw_model2 <- durbinWatsonTest(model2, alternative = "positive")
print(dw_model2)
```

```
##  lag Autocorrelation D-W Statistic p-value
##    1      0.03957229      1.852409   0.227
##  Alternative hypothesis: rho > 0
```

Model 1: $H \sim P$

D-W Statistic: 0.6208403.

P-value: 0 (or extremely close to 0).

Conclusion: The D-W statistic of 0.62 is far below the ideal value of 2 (which indicates no autocorrelation)3. The resulting P-value is highly significant ($P \approx 0$). We strongly reject the null hypothesis ($H_0 : \rho = 0$). This confirms the presence of strong positive first-order autocorrelation in the errors of Model 1, aligning with the visual evidence from the Time Plot and ACF Plot.

Model 2: $H \sim P + D$

D-W Statistic: 1.852409.

P-value: 0.227.

Conclusion: The D-W statistic of 1.85 is very close to 2, and the P-value of 0.227 is not statistically significant. We do not reject the null hypothesis ($H_0 : \rho = 0$). This indicates that first-order positive autocorrelation has been successfully eliminated from the residuals of Model 2.

Question 3

Multicollinearity (mtcars)

Use:

```r
dat <- mtcars
m_full <- lm(mpg ~ disp + hp + wt + qsec, data = dat)
```

(a) Report correlation matrix among numeric predictors. Identify |r|>0.8.

```r
predictors <- dat[, c("disp", "hp", "wt", "qsec")]
cor_matrix <- cor(predictors)
print("Correlation Matrix:")
```

```
## [1] "Correlation Matrix:"
```

```
print(round(cor_matrix, 4))
```

```
##         disp      hp      wt    qsec
## disp  1.0000  0.7909  0.8880 -0.4337
## hp    0.7909  1.0000  0.6587 -0.7082
## wt    0.8880  0.6587  1.0000 -0.1747
## qsec -0.4337 -0.7082 -0.1747  1.0000
```

```
strong_correlations <- cor_matrix
strong_correlations[abs(strong_correlations) <= 0.8] <- NA
diag(strong_correlations) <- NA
print(strong_correlations)
```

```
##           disp hp        wt qsec
## disp        NA NA 0.8879799   NA
## hp          NA NA        NA   NA
## wt   0.8879799 NA        NA   NA
## qsec        NA NA        NA   NA
```

The correlation matrix shows a very high correlation between disp (displacement) and wt (weight), with $r \approx 0.888$.

(b) Compute VIF; identify moderate (VIF>5) and severe (VIF>10) collinearity.

```
vif_values <- vif(m_full)
print("Variance Inflation Factors (VIF):")
```

```
## [1] "Variance Inflation Factors (VIF):"
```

```
print(round(vif_values, 2))
```

```
## disp   hp   wt qsec
## 7.99 5.17 6.92 3.13
```

The VIF measures how much the variance of an estimated regression coefficient is inflated due to collinearity. Disp has a VIF of 7.99, which is moderate. Wt has a VIF of 6.92, also classified as moderate.

(c) Fit a reduced model to mitigate collinearity; compare adjusted R^2 and SE of disp.

```
m_reduced <- lm(mpg ~ disp + hp + qsec, data = dat)

adj_r2_full <- summary(m_full)$adj.r.squared
adj_r2_reduced <- summary(m_reduced)$adj.r.squared
print(paste("Adjusted R-squared (Full):", round(adj_r2_full, 4)))
```

```
## [1] "Adjusted R-squared (Full): 0.8107"
```

```
print(paste("Adjusted R-squared (Reduced):", round(adj_r2_reduced, 4)))
```

```
## [1] "Adjusted R-squared (Reduced): 0.7279"
```

```
se_disp_full <- summary(m_full)$coefficients["disp", "Std. Error"]
se_disp_reduced <- summary(m_reduced)$coefficients["disp", "Std. Error"]
print(paste("SE of disp (Full Model):", round(se_disp_full, 4)))
```

```
## [1] "SE of disp (Full Model): 0.0107"
```

```
print(paste("SE of disp (Reduced Model):", round(se_disp_reduced, 4)))
```

```
## [1] "SE of disp (Reduced Model): 0.0078"
```

The adjust R^2 for the full model is larger than the reduced model. This might be because mt can more accurately reprecent mpg than disp though they are highly related. The SE of disp decreased after removing wt, leading to a more precise and reliable coefficient estimate for displacement.

Question 4

Model Selection with Subset Selection and Regularization

Using the built-in dataset mtcars, we will predict the miles per gallon (mpg) using all other variables as predictors.

(a) Split the data into training (70%) and test (30%) sets, using set.seed(9159); train_index_mtcars <- sample(1:nrow(mtcars), 0.7*nrow(mtcars)).

```
set.seed(9159)
dat <- mtcars
n <- nrow(dat)
train_index_mtcars <- sample(1:n, 0.7 * n)
train_data <- dat[train_index_mtcars, ]
test_data <- dat[-train_index_mtcars, ]
train_x <- as.matrix(train_data[, -1])
train_y <- train_data$mpg
test_x <- as.matrix(test_data[, -1])
test_y <- test_data$mpg
```

(b) Perform best subset selection on the training set and identify the model with the lowest BIC. Report the selected variables and the test MSE. (try using regsubsets function from leaps package)

```
sub_fit <- regsubsets(mpg ~ ., data = train_data, nvmax = 10, method = "exhaustive")
sub_summary <- summary(sub_fit)
bic_scores <- sub_summary$bic
best_model_bic_index <- which.min(bic_scores)
min_bic_score <- bic_scores[best_model_bic_index]
selected_vars_bic <- names(which(sub_summary$which[best_model_bic_index, ] == TRUE))
selected_predictors_bic <- selected_vars_bic[!selected_vars_bic %in% "(Intercept)"]

formula_str <- paste("mpg ~", paste(selected_predictors_bic, collapse = " + "))
best_model_bic <- lm(as.formula(formula_str), data = train_data)
predictions_bic <- predict(best_model_bic, newdata = test_data)
test_mse_bic <- mean((test_data$mpg - predictions_bic)^2)
print(paste("Selected Variables:", paste(selected_predictors_bic, collapse = ", ")))
```

```
## [1] "Selected Variables: wt, qsec"
```

```
print(paste("Minimum BIC Score (Training Set):", round(min_bic_score, 4)))
```

```
## [1] "Minimum BIC Score (Training Set): -31.4923"
```

```
print(paste("Test MSE:", round(test_mse_bic, 4)))
```

```
## [1] "Test MSE: 7.9823"
```

The model with highest BIC selects variables wt and qsec. Minimum BIC score is -31.4923 and the test MSE is 7.9823.

(c) Fit ridge and lasso regression models on the training set using cross-validation to select optimal lambdas. Report the selected variables (for lasso) and test MSEs for both models. (try using cv.glmnet function from glmnet package)

```
cv_ridge <- cv.glmnet(train_x, train_y, alpha = 0, nfolds = 10)
```

```
## Warning: Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per
## fold
```

```
best_lambda_ridge <- cv_ridge$lambda.min
ridge_model <- glmnet(train_x, train_y, alpha = 0, lambda = best_lambda_ridge)
predictions_ridge <- predict(ridge_model, newx = test_x)
test_mse_ridge <- mean((test_y - predictions_ridge)^2)
ridge_coefficients <- predict(ridge_model, type = "coefficients")[, 1]
selected_ridge_vars <- names(ridge_coefficients[ridge_coefficients != 0])
selected_ridge_predictors <- selected_ridge_vars[!selected_ridge_vars %in% "(Intercept)"
cv_lasso <- cv.glmnet(train_x, train_y, alpha = 1, nfolds = 10)
```

```
## Warning: Option grouped=FALSE enforced in cv.glmnet, since < 3 observations per
## fold
```

```
best_lambda_lasso <- cv_lasso$lambda.min
lasso_model <- glmnet(train_x, train_y, alpha = 1, lambda = best_lambda_lasso)
predictions_lasso <- predict(lasso_model, newx = test_x)
test_mse_lasso <- mean((test_y - predictions_lasso)^2)
lasso_coefficients <- predict(lasso_model, type = "coefficients")[, 1]
selected_lasso_vars <- names(lasso_coefficients[lasso_coefficients != 0])
selected_lasso_predictors <- selected_lasso_vars[!selected_lasso_vars %in% "(Intercept)"
print(paste("Ridge Optimal Lambda:", round(best_lambda_ridge, 4)))
```

```
## [1] "Ridge Optimal Lambda: 3.8398"
```

```r
print(paste("Ridge Test MSE:", round(test_mse_ridge, 4)))
```

```
## [1] "Ridge Test MSE: 3.1724"
```

```r
print(paste("Ridge Selected Variables (All Predictors Retained):",
            paste(selected_ridge_predictors, collapse = ", ")))
```

```
## [1] "Ridge Selected Variables (All Predictors Retained): cyl, disp, hp, drat, wt, qse
```

```r
print(paste("Lasso Optimal Lambda:", round(best_lambda_lasso, 4)))
```

```
## [1] "Lasso Optimal Lambda: 0.4023"
```

```r
print(paste("Lasso Selected Variables (Sparsity):",
           paste(selected_lasso_predictors, collapse = ", ")))
```

```
## [1] "Lasso Selected Variables (Sparsity): cyl, hp, drat, wt, qsec, carb"
```

```r
print(paste("Lasso Test MSE:", round(test_mse_lasso, 4)))
```

```
## [1] "Lasso Test MSE: 4.6189"
```

For Ridge, the optimal λ is 3.8398. Under the optimal λ, the test MSE is 3.1724 with all the variables selected.

For Lasso, the optimal λ is 0.4023. Under the optimal λ, the test MSE is 4.6189 with six variables selected.

Comparing teat MSE, the Ridge model achieved better prediction accuracy than the Lasso model. But Lasso model achieved a much simpler model by reducing the predictors from 10 to 6.